UML et modélisation des systèmes

Informatique — Licence 2 (L2)

Cours INF-L2-06

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours
Apprendre à décrire un logiciel avant de le construire : cas d'utilisation, diagramme de classes, diagrammes de séquence et de communication, diagrammes d'activité et d'états-transitions, puis passage du modèle au code.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Introduction à la modélisation et au langage UML
• Leçon 2 — Le diagramme de cas d'utilisation
• Leçon 3 — Le diagramme de classes : associations et multiplicités
• Leçon 4 — Diagrammes de séquence et de communication
• Leçon 5 — Diagrammes d'activité et d'états-transitions
• Leçon 6 — Du modèle UML au code : générer les squelettes de classes
• Leçon 7 — Projet : modéliser complètement une application
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

En L1, tu as appris à programmer : écrire des algorithmes, puis du code qui fonctionne. Mais dès qu'une application grandit — plusieurs écrans, plusieurs dizaines de classes, plusieurs développeurs — coder « au feeling » devient dangereux : chacun comprend le besoin à sa manière, les morceaux ne s'emboîtent plus, et les erreurs se découvrent tard, quand elles coûtent cher à corriger. Avant de construire une maison, l'architecte dessine des plans ; avant de construire un logiciel sérieux, l'informaticien dessine des modèles. Ces plans ne sont pas une perte de temps : ils servent à discuter avec le client, à repérer les oublis, à se répartir le travail entre développeurs et à garder une trace écrite des décisions prises. Dans les projets réels, la plupart des échecs ne viennent pas du code lui-même, mais d'un besoin mal compris dès le départ — exactement ce que la modélisation cherche à éviter.

UML, pour Unified Modeling Language, est le langage graphique standard de cette modélisation. Ce n'est ni un langage de programmation, ni une méthode de travail imposée : c'est une boîte de notations précises — des diagrammes — reconnue dans le monde entier et normalisée par l'OMG (Object Management Group). Avec UML, un diagramme dessiné à Bamako se lit sans difficulté à Dakar, à Montréal ou à Bangalore : c'est un langage commun pour penser et discuter un logiciel avant de l'écrire. Cette standardisation a une conséquence pratique énorme pour toi : les diagrammes appris ici sont les mêmes dans toutes les entreprises et dans tous les pays, quel que soit l'outil utilisé pour les dessiner. Les recruteurs le savent, et savoir lire ou produire un diagramme de classes propre fait partie des compétences de base attendues d'un

développeur, même débutant.

Ce cours te fait pratiquer les diagrammes les plus utiles du quotidien d'un développeur : les cas d'utilisation pour dire qui fait quoi avec le système, le diagramme de classes pour organiser les données et les responsabilités, les diagrammes de séquence et de communication pour décrire les échanges entre objets, le diagramme d'activité pour les processus, et le diagramme d'états-transitions pour le cycle de vie d'un objet. Un fil rouge unique nous suivra : « BiblioIAM », l'application de gestion de la bibliothèque de notre institut, que nous modéliserons de bout en bout, jusqu'à générer ses squelettes de classes en Java. À la fin du cours, tu sauras donc mener la modélisation complète d'une petite application — de l'idée de départ exprimée en français jusqu'aux squelettes de code — et surtout vérifier que tous tes diagrammes racontent bien la même histoire, sans se contredire.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Expliquer pourquoi on modélise un système avant de le coder, et distinguer les grandes familles de diagrammes UML.
- Identifier les acteurs d'une application et construire un diagramme de cas d'utilisation avec relations d'inclusion, d'extension et de généralisation.
- Construire un diagramme de classes complet : attributs, opérations, visibilités et classes abstraites.
- Placer des multiplicités correctes et justifiées sur les associations.
- Choisir entre association simple, agrégation, composition, généralisation et dépendance selon la situation.
- Décrire un scénario avec un diagramme de séquence (messages synchrones et asynchrones, fragments alt/opt/loop) et sa version en diagramme de communication.
- Modéliser un processus avec un diagramme d'activité et le cycle de vie d'un objet avec un diagramme d'états-transitions, puis traduire un diagramme de classes en squelettes de code.

Leçon 1 — Introduction à la modélisation et au langage UML

Modéliser, c'est construire une représentation simplifiée d'un système pour mieux le comprendre et le faire comprendre. Un modèle n'est pas le système lui-même : il retient seulement ce qui est utile pour répondre à une question donnée. Le plan d'une maison ne montre ni la couleur des murs ni la marque des robinets, mais il suffit pour décider de la place des pièces et en discuter avec le client. De la même façon, un modèle de logiciel aide à valider les besoins avec l'utilisateur, à repérer les contradictions tôt, à répartir le travail entre développeurs et à documenter l'application pour ceux qui la feront évoluer. Le modèle joue aussi le rôle de mémoire du projet : six mois après la livraison, quand il faudra corriger un bug ou ajouter une fonctionnalité, c'est lui qu'on relira pour comprendre pourquoi les choses ont été organisées ainsi, bien avant de plonger dans des milliers de lignes de code.

UML est né dans les années 1990 de la fusion des notations de trois spécialistes — Grady Booch, James Rumbaugh et Ivar Jacobson, surnommés les « trois amigos » — puis a été adopté comme standard par l'OMG. La version 2 du langage compte quatorze diagrammes officiels, mais personne ne les utilise tous sur un même projet : on choisit ceux qui répondent aux questions du moment. Attention à ne pas confondre : UML dit COMMENT dessiner, pas DANS QUEL ORDRE ni AVEC QUELLE méthode travailler. C'est une notation, pas un mode d'emploi de projet. Connaître cette histoire aide à comprendre l'esprit du langage : UML n'est pas tombé d'un comité lointain, il est né de la pratique de terrain, en mettant fin à la « guerre des méthodes » qui voyait chaque consultant défendre sa propre notation incompatible avec celle des autres.

Les quatorze diagrammes se rangent en deux familles. Les diagrammes structurels décrivent le système au repos : de quoi il est composé (classes, objets, composants, déploiement…). Les diagrammes comportementaux décrivent le système en mouvement : ce qu'il fait (cas d'utilisation), comment les objets se parlent dans le temps (séquence, communication), comment un processus avance (activité), et comment un objet change d'état au fil de sa vie (états-transitions). Dans ce cours, nous étudierons les six diagrammes les plus utilisés en pratique, et surtout nous apprendrons à les rendre cohérents entre eux : ce sont des vues différentes d'un même système, jamais des dessins isolés. Dans un projet bien mené, on ne dessine d'ailleurs jamais les quatorze diagrammes : on choisit en fonction des questions ouvertes du moment, et l'on arrête de modéliser dès que la réponse devient claire — modéliser pour le plaisir de modéliser est une perte de temps aussi réelle que coder sans réfléchir.

L'essentiel
Un modèle simplifie un système pour le comprendre, le discuter et le construire sans erreur coûteuse. UML est la notation graphique standard (OMG) pour ces modèles : 14 diagrammes en deux familles, structure et comportement. UML n'est ni un langage de programmation ni une méthode : il dit comment dessiner, pas comment mener le projet.

Exemple — Les six diagrammes du cours et la question à laquelle chacun répond

```
Diagramme Question à laquelle il répond
--------------------- ---------------------------------------------
Cas d'utilisation Qui utilise le système, et pour faire quoi ?
Classes De quelles données et responsabilités
le système est-il composé ?
Séquence Dans quel ordre les objets échangent-ils
leurs messages pendant un scénario ?
Communication Quels objets sont reliés, et que se
disent-ils (même scénario, autre vue) ?
Activité Comment un processus avance-t-il,
étape par étape, choix compris ?
États-transitions Par quels états passe UN objet pendant
toute sa vie, et pourquoi change-t-il ?

Règle d'or : un même système, plusieurs vues,
mais toujours UNE seule vérité — les vues
doivent rester cohérentes entre elles.
```

À retenir

- Modéliser sert à comprendre, communiquer et décider AVANT de coder : corriger un schéma coûte cent fois moins cher que corriger un programme livré.
- UML = une notation standardisée par l'OMG, pas un langage de programmation et pas une méthode de projet.
- Diagrammes structurels (le système au repos) et diagrammes comportementaux (le système en mouvement) sont des vues complémentaires d'un même système.

Exercice

Pour le futur logiciel de la cantine scolaire de l'IAM, un collègue te demande deux choses : (1) « je veux voir qui utilisera le logiciel et ce que chacun pourra y faire » ; (2) « je veux savoir comment sont organisées les données : les élèves, les plats, les tickets ». Pour chaque demande, dis quel diagramme étudié dans ce cours tu dessinerais, et explique pourquoi un diagramme de séquence serait prématuré à ce stade.

✔ Corrigé
Demande (1) : le diagramme de cas d'utilisation — il montre les acteurs (élève, cuisinier, caissier, responsable) et les fonctionnalités offertes à chacun, sans entrer dans le fonctionnement interne. Demande (2) : le diagramme de classes — il décrit les concepts du domaine (Élève, Plat, Ticket) avec leurs attributs et leurs liens. Un diagramme de séquence serait prématuré car il raconte le déroulement précis d'UN scénario, message par message : tant qu'on n'a pas stabilisé ni les fonctionnalités (cas d'utilisation) ni les objets en présence (classes), on ne sait même pas quels objets échangeraient ces messages. L'ordre de travail naturel est : d'abord le « qui fait quoi », puis le « avec quoi », enfin le « dans quel ordre ».

Leçon 2 — Le diagramme de cas d'utilisation

Le diagramme de cas d'utilisation décrit les fonctionnalités offertes par le système, vues de l'extérieur, du point de vue de ceux qui s'en servent. On y trouve d'abord les acteurs : tout ce qui est EXTERNE au système et interagit avec lui — un rôle humain (le bibliothécaire, l'étudiant) mais aussi un autre système (un service de paiement en ligne). Un acteur est un RÔLE, pas une personne précise : « Bibliothécaire » est un acteur, « Madame Diarra » ne l'est pas. On distingue souvent l'acteur principal, qui initie le cas, des acteurs secondaires, sollicités par le système. Le tout est enfermé dans un rectangle qui marque la frontière du système : dedans, ce qu'on construit ; dehors, ceux qui l'utilisent.

Chaque cas d'utilisation est une fonctionnalité complète qui apporte un résultat observable à un acteur : il se nomme par un verbe suivi d'un complément — « Emprunter un livre », « Rechercher un livre », « Gérer les rayons » — et se dessine par une ellipse reliée à ses acteurs par un trait plein. Les acteurs eux-mêmes peuvent se spécialiser : si les étudiants et les enseignants empruntent les livres exactement de la même façon, on crée un acteur général « Adhérent », relié aux deux acteurs spécialisés par une flèche de généralisation (triangle vide pointant vers l'acteur général). Étudiant et Enseignant héritent alors de toutes les associations d'Adhérent : c'est le même mécanisme d'héritage que pour les classes, appliqué aux rôles.

Deux relations permettent de factoriser les cas. L'INCLUSION (<<include>>, flèche pointillée du cas de base vers le cas inclus) signifie que le cas inclus se produit TOUJOURS pendant le cas de base : « Emprunter un livre » inclut « S'authentifier » et « Vérifier la disponibilité », car aucun emprunt ne se fait sans ces étapes — on les extrait pour ne pas les répéter dans chaque cas. L'EXTENSION (<<extend>>, flèche pointillée du cas d'extension vers le cas de base) signifie un comportement OPTIONNEL, inséré seulement si une condition est remplie : « Prolonger un emprunt » étend « Consulter mes emprunts », mais seulement si l'emprunt n'est pas en retard. Retiens le critère décisif : toujours = include, parfois sous condition = extend, et le sens des flèches est inverse dans les deux cas.

L'essentiel
Le diagramme de cas d'utilisation montre les acteurs (des rôles externes au système) et les fonctionnalités (verbe + complément) qu'ils utilisent, dans la frontière du système. <<include>> = étape obligatoire et toujours présente, flèche du cas de base vers le cas inclus. <<extend>> = comportement optionnel sous condition, flèche du cas d'extension vers le cas de base.

Exemple — Diagramme de cas d'utilisation de BiblioIAM (texte structuré)

```
Système : BiblioIAM (gestion de la bibliothèque)

ACTEURS
général : Adhérent
spécialisés: Étudiant ----▷ Adhérent
Enseignant --▷ Adhérent
principal : Bibliothécaire

CAS D'UTILISATION (dans la frontière du système)
- Rechercher un livre : Adhérent, Bibliothécaire
- Emprunter un livre : Adhérent
- Retourner un livre : Adhérent
- Consulter mes emprunts : Adhérent
- Gérer les livres et rayons : Bibliothécaire
- Enregistrer un retour : Bibliothécaire

RELATIONS ENTRE CAS
Emprunter un livre
--- <<include>> ---> S'authentifier
--- <<include>> ---> Vérifier la disponibilité
Prolonger un emprunt
--- <<extend>> ----> Consulter mes emprunts
condition : [emprunt non en retard]
point d'extension : depuis l'écran des emprunts
Réserver un livre
--- <<extend>> ----> Rechercher un livre
condition : [livre indisponible]
```

À retenir

• Un acteur est un rôle externe au système, jamais une personne nommée ni un élément interne du logiciel.

• Include = toujours et obligatoirement ; la flèche part du cas de base vers le cas inclus.

• Extend = parfois, sous condition explicite ; la flèche part du cas d'extension vers le cas de base.

Exercice

Dans BiblioIAM, le retour d'un livre se fait au comptoir. Si le livre revient en retard, l'adhérent doit payer une pénalité avant que le retour soit validé ; s'il revient à temps, il ne paie rien. Un étudiant propose de relier « Payer la pénalité » à « Retourner un livre » par un <<include>>. A-t-il raison ? Justifie, donne la bonne relation, le sens de la flèche et la condition à écrire.

✔ Corrigé
Non, il n'a pas raison. Un <<include>> signifierait que la pénalité est payée à CHAQUE retour, même pour un livre rendu à temps : ce serait absurde. La pénalité n'intervient que parfois, lorsqu'une condition est remplie (le retard) : c'est donc une EXTENSION. On dessine le cas « Payer la pénalité » avec une flèche pointillée <<extend>> QUI PART de « Payer la pénalité » et ARRIVE sur le cas de base « Retourner un livre », avec la condition [date de retour réelle postérieure à la date prévue] et le point d'extension situé au moment de l'enregistrement du retour. Test simple à garder : demande-toi « est-ce que ça arrive à tous les coups ? ». Oui → include ; seulement dans certains cas → extend.

Leçon 3 — Le diagramme de classes : associations et multiplicités

Le diagramme de classes est la pièce maîtresse de la modélisation objet : il décrit les concepts du domaine et leurs liens, indépendamment de tout scénario particulier. Une classe se dessine par un rectangle à trois compartiments : le nom en haut, les attributs au milieu, les opérations en bas. Chaque membre porte un symbole de visibilité : + public (accessible partout), − privé (accessible seulement dans la classe), # protégé (accessible dans la classe et ses sous-classes). Les attributs sont typés (− titre : String), les opérations signées (+ emprunter() : boolean). Une classe ABSTRAITE — qu'on ne peut pas instancier directement — s'écrit en italique ou avec la mention {abstract} : chez nous, Personne est abstraite, car tout adhérent réel est forcément un Étudiant ou un Enseignant.

Les classes sont reliées par des associations, qui représentent les liens durables entre objets. Sur chaque extrémité, on écrit une MULTIPLICITÉ : le nombre d'objets de l'autre côté pouvant être liés à un objet d'ici. Les valeurs usuelles sont 1 (exactement un), 0..1 (zéro ou un), 0..* ou * (plusieurs, zéro compris) et 1..* (au moins un). La lecture se fait dans les deux sens : « une Personne réalise de 0 à plusieurs Emprunts » et « un Emprunt est réalisé par exactement 1 Personne ». Quand le lien porte lui-même des informations — les dates d'un emprunt ne sont ni au livre seul ni à l'adhérent seul — on en fait une vraie classe, comme notre classe Emprunt, plutôt qu'une simple ligne.

Quatre relations spéciales complètent la boîte à outils, et les confondre est l'erreur d'examen la plus fréquente. La GÉNÉRALISATION (flèche à triangle vide vers la classe mère) exprime un « est-un » durable : un Étudiant EST une Personne, il en hérite attributs et opérations. L'AGRÉGATION (losange VIDE du côté du tout) exprime un « a-un » faible : les parties existent indépendamment du tout et survivent à sa disparition. La COMPOSITION (losange PLEIN du côté du tout) est un « a-un » fort : la partie n'appartient qu'à un seul tout à la fois et est détruite avec lui — les rayons de NOTRE bâtiment bibliothèque n'ont de sens que dans cette bibliothèque. Enfin la DÉPENDANCE (flèche pointillée) signale une utilisation ponctuelle : le Bibliothécaire crée des Emprunts au comptoir, mais ne les « contient » pas en permanence.

L'essentiel
Une classe = nom, attributs typés, opérations signées, avec visibilités (+ public, − privé, # protégé). Les multiplicités (1, 0..1, 0..*, 1..*) se lisent dans les deux sens et doivent être justifiées par les règles du domaine. Généralisation = « est-un » hérité ; agrégation = « a-un » faible (losange vide, parties indépendantes) ; composition = « a-un » fort (losange plein, partie détruite avec le tout) ; dépendance = usage ponctuel (pointillé).

Exemple — Diagramme de classes de BiblioIAM (texte structuré)

```
CLASSES (visibilité : + public, - privé, # protégé)
{abstract} Personne
- nom : String ; - prenom : String ; - email : String
+ sePresenter() : String {abstract}
Etudiant extends Personne
- numeroCarte : String ; - filiere : String
+ peutEmprunter() : boolean
Enseignant extends Personne
- departement : String
Livre
- isbn : String ; - titre : String ; - auteur : String
- nbExemplaires : int
+ estDisponible() : boolean
Emprunt
- dateEmprunt : Date ; - dateRetourPrevue : Date
- dateRetour : Date (vide tant que le livre n'est pas rendu)
+ estEnRetard() : boolean ; + clore(date : Date)
Bibliothecaire
- matricule : String
+ enregistrerEmprunt() ; + enregistrerRetour()
Rayon
- nom : String
Bibliotheque
- nom : String

RELATIONS
Personne 1 ------- 0..* Emprunt (réalise)
un emprunt est fait par exactement UN adhérent ;
un adhérent peut n'avoir aucun emprunt en cours.
Livre 1 ------- 0..* Emprunt (concerne)
un emprunt porte sur exactement UN livre
(simplification assumée : un emprunt = un livre).
Bibliotheque 1 ◆--- 1..* Rayon COMPOSITION
les rayons n'existent que dans cette bibliothèque
et disparaissent avec elle.
Rayon 1 ------- 0..* Livre (range)
chaque livre est rangé dans exactement UN rayon ;
si le rayon est supprimé, le livre est juste déplacé
vers un autre rayon : association simple, pas composition.
Bibliothecaire - - -> Emprunt DÉPENDANCE (crée)
le bibliothécaire crée les emprunts au comptoir
sans les contenir durablement.
```

À retenir

• Lis chaque association dans les deux sens et justifie chaque multiplicité par une phrase du domaine (« un emprunt est fait par exactement un adhérent »).

• Composition (losange plein) : la partie n'a qu'un seul tout et meurt avec lui ; agrégation (losange vide) : la partie mène sa vie indépendante.

• Si un lien porte des données propres (dates, quantités), transforme-le en classe : c'est le cas d'Emprunt entre Personne et Livre.

Exercice

Un petit site de vente en ligne manipule les concepts suivants : Client, Commande, LigneCommande (un article commandé, avec quantité et prix au moment de l'achat) et Produit. Propose les associations entre ces quatre classes avec leurs multiplicités, en précisant pour chacune s'il s'agit d'une association simple, d'une agrégation ou d'une composition, et justifie chaque choix en une phrase.

✔ Corrigé
Client 1 — 0..* Commande (association simple) : une commande est passée par exactement un client ; un client peut n'avoir encore rien commandé. Commande 1 ◆— 1..* LigneCommande (COMPOSITION) : une ligne n'existe que dans sa commande, elle n'est partagée par aucune autre commande, et supprimer la commande supprime ses lignes ; une commande contient au moins une ligne, sinon elle n'a pas de sens. LigneCommande * — 1 Produit (association simple) : chaque ligne désigne exactement un produit du catalogue, mais le produit, lui, survit à la commande (il reste au catalogue même si la commande est supprimée) — ce n'est donc PAS une composition côté produit. Erreur à éviter : mettre la multiplicité 0..* côté Produit sur l'association Client—Commande, ou dessiner la composition entre Commande et Produit, alors qu'elle concerne Commande et LigneCommande.

Leçon 4 — Diagrammes de séquence et de communication

Le diagramme de séquence raconte l'histoire d'UN scénario précis d'un cas d'utilisation, en montrant les échanges entre objets dans l'ordre chronologique. Les participants sont alignés en haut, chacun prolongé vers le bas par une ligne de vie en pointillés ; le temps s'écoule de haut en bas. Un message se dessine par une flèche horizontale portant le nom de l'opération appelée. Deux natures de messages doivent être distinguées sans faute : le message SYNCHRONE (pointe de flèche PLEINE) bloque l'émetteur, qui attend la réponse avant de continuer — c'est l'appel de méthode classique ; le message ASYNCHRONE (pointe de flèche OUVERTE, en demi-pointe) n'attend rien : l'émetteur envoie et poursuit son travail, comme lorsqu'on confie un e-mail à un service d'envoi. Les réponses se dessinent en flèches pointillées revenant vers l'appelant, et les fines barres rectangulaires sur les lignes de vie (barres d'activation) montrent pendant combien de temps chaque objet est occupé. Un message de création, noté « create », fait naître un objet en cours de scénario ; une croix en bout de ligne de vie marque sa destruction.

Un scénario réel n'est pas toujours linéaire : les FRAGMENTS COMBINÉS découpent le diagramme en zones encadrées avec un opérateur. « alt » exprime un choix entre plusieurs branches gardées par des conditions ([livre disponible] / [sinon]) ; « opt » un passage optionnel exécuté seulement si sa garde est vraie ; « loop » répète son contenu pour une collection ou tant qu'une condition tient ; « ref » renvoie à un autre diagramme décrit ailleurs. Grâce à eux, on peut montrer le scénario nominal et ses variantes sans dessiner dix diagrammes. Mais garde la mesure : un diagramme de séquence décrit un scénario, pas tout le système ; s'il faut trois pages pour le lire, c'est qu'il faut le découper en plusieurs scénarios.

Le diagramme de communication décrit exactement le même scénario, mais avec le point de vue inverse : on y voit d'abord QUI est relié à qui. Les objets sont disposés librement sur la page, reliés par les liens qui existent entre eux (ceux du diagramme de classes), et les messages sont portés le long de ces liens avec une NUMÉROTATION qui rétablit l'ordre temporel : 1, 2, 3 pour les messages de premier niveau, et 1.1, 1.2 lorsqu'un message déclenche des appels imbriqués pendant son exécution. Les deux diagrammes sont sémantiquement équivalents — la plupart des outils transforment l'un en l'autre automatiquement. Choisis la séquence pour montrer la chronologie et les alternatives, la communication pour montrer la structure des collaborations et repérer un objet « bavard » relié à tout le monde.

L'essentiel
Un diagramme de séquence = un scénario, des participants en colonnes, le temps de haut en bas. Message synchrone : flèche pleine, l'émetteur attend ; message asynchrone : flèche ouverte, l'émetteur continue ; réponse : flèche pointillée. Les fragments alt / opt / loop / ref structurent les variantes. Le diagramme de communication montre le même scénario par les liens, avec des messages numérotés (1, 1.1, 1.2…).

Exemple — Diagramme de séquence du scénario « Emprunter un livre »

```
Participants (de gauche à droite) :
:Bibliothécaire (acteur) | :InterfaceBiblio | :GestionEmprunts
| :Etudiant | :Livre | :Emprunt | :ServiceEmail

Messages (--> synchrone, - -> réponse pointillée, >> asynchrone) :
1. Bibliothécaire --> InterfaceBiblio
: saisirEmprunt(codeCarte, isbn)
2. InterfaceBiblio --> GestionEmprunts
: nouvelEmprunt(codeCarte, isbn)
3. GestionEmprunts --> Etudiant : rechercher(codeCarte)
3.1 GestionEmprunts - -> réponse : adhérent trouvé
4. GestionEmprunts --> Etudiant : peutEmprunter()
4.1 - -> réponse : vrai (moins de 3 emprunts en cours)
5. GestionEmprunts --> Livre : estDisponible()
5.1 - -> réponse : vrai
6. GestionEmprunts --> Emprunt : « create »
new Emprunt(dateEmprunt, dateRetourPrevue)
7. GestionEmprunts --> Livre : decrementerExemplaires()
8. GestionEmprunts >> ServiceEmail : envoyerConfirmation()
(asynchrone : on n'attend PAS l'envoi du mail
pour terminer l'emprunt au comptoir)
9. GestionEmprunts - -> InterfaceBiblio : reçu d'emprunt
10. InterfaceBiblio - -> Bibliothécaire : afficher(reçu)

Fragment [alt] autour des messages 5 à 9 :
[livre disponible] -> déroulement ci-dessus
[sinon] -> GestionEmprunts répond
« indisponible » et propose une réservation
```

À retenir

• Synchrone = flèche pleine + attente ; asynchrone = flèche ouverte + l'émetteur continue ; retour = pointillé. Les confondre inverse le sens du scénario.

• Un diagramme de séquence couvre UN scénario d'UN cas d'utilisation ; les fragments alt/opt/loop gèrent les variantes à l'intérieur de ce scénario.

• Séquence et communication disent la même chose : la première privilégie la chronologie, la seconde les liens entre objets (messages numérotés, imbrication par 1.1, 1.2…).

Exercice

Reprends le scénario « Retourner un livre » de BiblioIAM et décris-le sous forme de diagramme de COMMUNICATION : liste les objets participants et les liens utilisés, puis les messages avec la numérotation imbriquée correcte (1, 1.1, 1.2…). Le retour comprend : la saisie du code du livre par le bibliothécaire, la recherche de l'emprunt en cours, la vérification du retard, la clôture de l'emprunt et la remise du livre en rayon (incrémentation des exemplaires).

✔ Corrigé
Participants : :Bibliothécaire (acteur), :InterfaceBiblio, :GestionEmprunts, :Emprunt, :Livre. Liens : Bibliothécaire—InterfaceBiblio, InterfaceBiblio—GestionEmprunts, GestionEmprunts—Emprunt, GestionEmprunts—Livre, Emprunt—Livre (l'emprunt connaît son livre). Messages numérotés : 1 : saisirRetour(isbn) de Bibliothécaire vers InterfaceBiblio ; puis, imbriqués dans son exécution : 1.1 : enregistrerRetour(isbn) vers GestionEmprunts ; 1.1.1 : rechercherEmpruntEnCours(isbn) vers Emprunt (réponse : l'emprunt trouvé) ; 1.1.2 : estEnRetard() vers ce même Emprunt ; 1.1.3 : clore(dateDuJour) vers Emprunt ; 1.1.4 : incrementerExemplaires() vers Livre ; 1.2 : afficherConfirmation() en retour vers le Bibliothécaire. La logique de la numérotation est la clé du corrigé : un message reçoit le numéro de son déclencheur suivi d'un rang (1.1.2 est le deuxième appel déclenché PENDANT l'exécution de 1.1). En diagramme de séquence, les mêmes échanges se liraient de haut en bas, dans l'ordre 1, 1.1, 1.1.1… de haut en bas des lignes de vie.

Leçon 5 — Diagrammes d'activité et d'états-transitions

Le diagramme d'activité décrit un processus comme un enchaînement d'actions reliées par des flux de contrôle, un peu comme un algorithme dessiné. Les actions sont des rectangles aux coins arrondis, le départ est un point noir (nœud initial), la fin un point noir entouré d'un cercle (nœud final). Les choix se dessinent par un losange de décision dont chaque sortie porte une garde entre crochets — [livre disponible], [en retard] — et les chemins se rejoignent dans un losange de fusion. Deux barres spéciales gèrent le parallélisme : la FOURCHE (fork) éclate un flux en plusieurs flux simultanés, et la JONCTION (join) attend que TOUS les flux parallèles soient terminés avant de continuer. Enfin, les couloirs (swimlanes) partagent le diagramme en zones de responsabilité : ce que fait le bibliothécaire, ce que fait le système, ce que fait l'adhérent.

Le diagramme d'états-transitions change complètement de point de vue : au lieu d'un processus, il suit la vie d'UN SEUL objet, du berceau à la tombe. Un ÉTAT est une situation durable dans laquelle l'objet se trouve et réagit de façon caractéristique — un livre peut être Disponible, Emprunté, En retard, Réservé ou En réparation. Une TRANSITION relie deux états et se déclenche par un ÉVÉNEMENT, parfois sous une garde, parfois accompagné d'une action : on la note événement [garde] / action. « Emprunté —retourner→ Disponible » se lit : quand l'événement « retourner » survient, l'objet passe de l'état Emprunté à l'état Disponible. Certains événements sont temporels : un livre emprunté devient En retard automatiquement quand la date passe la date de retour prévue, sans que personne n'agisse.

Ne confonds plus les deux diagrammes : l'activité raconte un PROCESSUS qui peut traverser des dizaines d'objets et plusieurs acteurs ; les états-transitions racontent la BIOGRAPHIE d'un seul objet. Le test pour choisir : si ta question commence par « quelles sont les étapes pour… », c'est une activité ; si elle commence par « dans quelles situations cet objet peut-il se trouver, et qu'est-ce qui l'en fait sortir ? », c'est un diagramme d'états-transitions. En pratique, on ne dessine un diagramme d'états

que pour les objets au cycle de vie riche et contraint — Emprunt, Commande, Livre — jamais pour un objet banal qui naît, sert et disparaît sans règles.

L'essentiel
Diagramme d'activité : un processus — actions, décisions gardées [condition], fourche/jonction pour le parallélisme, couloirs pour les responsabilités. Diagramme d'états-transitions : la vie d'UN objet — états durables reliés par des transitions notées événement [garde] / action. Le premier répond à « quelles étapes ? », le second à « quelles situations possibles pour cet objet ? ».

Exemple — Diagramme d'états-transitions de l'objet Livre (BiblioIAM)

```
États de Livre : Disponible, Emprunté, En retard,
Réservé, En réparation

(•) = état initial : un nouveau livre entre au
catalogue directement dans l'état Disponible.

Transitions (événement [garde] / action) :
Disponible -- emprunter() --------------------> Emprunté
Emprunté -- retourner() --------------------> Disponible
Emprunté -- date du jour > dateRetourPrevue -> En retard
(événement temporel : automatique,
aucun acteur ne déclenche ce passage)
En retard -- retourner() / calculerPénalité --> Disponible
Disponible -- réserver() ----------------------> Réservé
Réservé -- retirerLivre() ------------------> Emprunté
Réservé -- after(3 jours) / libérer --------> Disponible
(réservation expirée sans retrait)
Disponible -- signalerEndommagé() ------------> En réparation
En réparation -- réparationTerminée() --------> Disponible

(◎) = état final : atteint par l'événement
retirerDuCatalogue(), possible depuis
Disponible ou En réparation seulement.

Lecture : depuis Emprunté, on NE PEUT PAS passer
directement à Réservé ni à En réparation :
le diagramme interdit ces chemins, et c'est
exactement ce qu'on attend d'un modèle.
```

À retenir

- Activité = processus et étapes ; états-transitions = biographie d'un objet. Le choix du diagramme dépend de la question posée.
- Une transition se lit : événement [garde] / action. Sans événement déclencheur, pas de changement d'état (sauf événement temporel explicite).
- Le diagramme d'états a une valeur d'interdiction : tout chemin non dessiné est impossible pour l'objet — c'est un contrat sur son cycle de vie.

Exercice

Dessine, en texte structuré comme dans l'exemple, le diagramme d'états-transitions d'un objet Commande dans une boutique en ligne, sachant que : une commande naît « En attente de paiement » ; le paiement la fait passer à « Payée » ; l'atelier la prépare puis l'expédie (« Expédiée ») ; le transporteur la livre (« Livrée », état final) ; le client peut annuler tant qu'elle n'est pas expédiée (« Annulée », état final) ; un paiement refusé ramène la commande en attente. Précise l'événement de chaque transition.

✔ Corrigé
États : En attente de paiement (initial), Payée, Expédiée, Livrée (final), Annulée (final). Transitions : En attente —paiementAccepté→ Payée ; En attente —paiementRefusé→ En attente (boucle sur place : la commande reste en attente, le client peut réessayer) ; Payée —expédier→ Expédiée ; Expédiée —livrer→ Livrée ; En attente —annuler→ Annulée ; Payée —annuler / rembourser→ Annulée. Le point délicat est l'annulation : elle est possible depuis En attente ET depuis Payée (avec l'action / rembourser), mais il n'existe AUCUNE transition annuler depuis Expédiée ou Livrée — le diagramme rend donc l'annulation tardive impossible, ce qui traduit exactement la règle métier donnée. De même, aucun événement ne sort de Livrée ni d'Annulée : ce sont des états finaux, la vie de la commande est terminée.

Leçon 6 — Du modèle UML au code : générer les squelettes de classes

Le diagramme de classes n'est pas un dessin décoratif : il se traduit en code de façon presque mécanique, et c'est ce qui en fait la valeur. Les règles de base tiennent en quelques lignes. Une classe UML devient une classe du langage ; un attribut typé devient un champ, et sa visibilité se conserve : le − UML devient private, le + devient public, le # devient protected. Une opération devient une méthode avec sa signature. Une classe abstraite devient une classe abstract, qu'on ne peut pas instancier, et la généralisation devient un héritage (extends en Java). Les types UML se convertissent dans le langage cible : String reste String, Date devient LocalDate en Java moderne, les booléens et entiers gardent leur sens.

Le vrai travail concerne les associations, car un lien UML ne dit pas comment le code stocke la relation. La convention la plus répandue : une association de multiplicité 1 d'une classe A vers une classe B devient, dans A, un attribut de type B (une simple référence) ; une association de multiplicité * devient une COLLECTION, en général une List<B>. Si l'association est navigable dans les deux sens, on place un attribut de chaque côté, et il faut alors des méthodes qui maintiennent les deux côtés cohérents (ajouter un emprunt à l'adhérent doit aussi l'attacher au livre). La COMPOSITION impose davantage : c'est le tout qui crée ses parties — typiquement dans son constructeur — et on ne les partage pas avec l'extérieur, afin que leur durée de vie reste liée à celle du tout. La dépendance, elle, ne crée aucun attribut : elle se traduit par un simple paramètre de méthode ou une création locale, exactement comme le Bibliothécaire qui fabrique un Emprunt le temps de l'enregistrement.

Beaucoup d'outils (modeleurs UML, plugins d'IDE) génèrent automatiquement ces squelettes depuis le diagramme, et savent aussi faire le chemin inverse — reconstruire un diagramme depuis du code existant (rétro-ingénierie). Mais ne tombe pas dans le

piège symétrique : le code généré ne contient que la structure, jamais la logique métier, et il perd certaines finesses du modèle (les gardes, les justifications de multiplicités, les contraintes du type « au plus 3 emprunts »). Le modèle reste donc un document vivant, à faire évoluer avec le code. Retiens la compétence visée : étant donné un diagramme de classes, tu dois pouvoir écrire le squelette à la main, proprement, et réciproquement lire un squelette et y reconnaître le modèle d'origine.

L'essentiel
Traduction mécanique : classe → classe, attribut visibilité comprise → champ, opération → méthode, généralisation → extends, classe abstraite → abstract. Association vers multiplicité 1 → attribut référence ; vers multiplicité * → collection (List). Composition → le tout crée ses parties (constructeur) ; dépendance → simple paramètre ou création locale, jamais d'attribut. Le squelette reproduit la structure, pas la logique métier.

Exemple — Squelette Java EXACTEMENT déduit du diagramme de classes de la leçon 3

```
import java.time.LocalDate;
import java.util.*;
public abstract class Personne {
private String nom, prenom, email;
private List<Emprunt> emprunts = new ArrayList<>(); // 1 --- 0..*
public abstract String sePresenter();
public List<Emprunt> getEmprunts() { return emprunts; }
}
class Etudiant extends Personne {
private String numeroCarte, filiere;
public boolean peutEmprunter() { return getEmprunts().size() < 3; }
public String sePresenter() { return "Etudiant " + numeroCarte; }
}
class Enseignant extends Personne {
private String departement;
public String sePresenter() { return "Enseignant " + departement; }
}
class Livre {
private String isbn, titre, auteur;
private int nbExemplaires;
private Rayon rayon; // rangé dans exactement UN rayon
private List<Emprunt> emprunts = new ArrayList<>();
public boolean estDisponible() { return nbExemplaires > 0; }
}
class Emprunt {
private LocalDate dateEmprunt, dateRetourPrevue, dateRetour;
private Personne adherent; // multiplicité 1
private Livre livre; // multiplicité 1
public boolean estEnRetard() { return dateRetour == null
&& LocalDate.now().isAfter(dateRetourPrevue); }
public void clore(LocalDate d) { this.dateRetour = d; }
}
class Rayon { private String nom;
private List<Livre> livres = new ArrayList<>();
public void ranger(Livre l) { livres.add(l); }
}
class Bibliotheque {
private String nom;
private List<Rayon> rayons = new ArrayList<>(); // COMPOSITION
public Bibliotheque() { rayons.add(new Rayon()); } // créés par le tout
}
class Bibliothecaire {
private String matricule; // DÉPENDANCE : crée sans contenir
public Emprunt enregistrerEmprunt(Personne p, Livre l) { return new Emprunt(); }
}
```

À retenir

• Multiplicité 1 → attribut simple du type cible ; multiplicité * → collection (List) ; association bidirectionnelle → un attribut de chaque côté, à synchroniser.

• La composition se voit dans le code : les parties naissent dans le tout (constructeur) et ne sont pas partagées ; la dépendance ne laisse aucune trace en attribut.

• Vérification croisée obligatoire : chaque classe du diagramme a sa classe dans le code, chaque attribut y figure avec la même visibilité, chaque association est

traduite.

Exercice

Voici un extrait de diagramme de classes d'une application de transport : Bus { − immatriculation : String ; − capacite : int } ; Chauffeur { − nom : String ; − permis : String } ; association « conduire » : un Chauffeur conduit de 0 à plusieurs Bus au cours de sa carrière, et chaque Bus est conduit par exactement un Chauffeur à la fois ; Bus 1 ◆— 1 Moteur (composition, un bus possède exactement un moteur qui lui est propre). Écris le squelette Java correspondant (champs et en-têtes de méthodes seulement) en justifiant la traduction de chaque relation.

✔ Corrigé

Squelette attendu : class Chauffeur { private String nom, permis; private List<Bus> busConduits = new ArrayList<>(); } — la multiplicité 0..* côté Bus donne une collection dans Chauffeur. class Bus { private String immatriculation; private int capacite; private Chauffeur chauffeurActuel; private Moteur moteur; public Bus() { this.moteur = new Moteur(); } } — la multiplicité 1 côté Chauffeur donne un simple attribut référence dans Bus ; la composition 1 ◆— 1 avec Moteur donne un attribut moteur créé dans le constructeur de Bus (le tout fabrique sa partie, qui ne sera partagée avec aucun autre bus). class Moteur { } suffit comme coquille. Justifications : * → List ; 1 → référence simple ; composition → création dans le constructeur et pas de setter public qui permettrait de « prêter » le moteur à un autre bus. Si ton squelette met une List<Moteur> dans Bus ou crée le moteur à l'extérieur puis le passe en paramètre, la multiplicité 1 et la composition ne sont pas respectées.

Leçon 7 — Projet : modéliser complètement une application

Mettre tout ensemble demande une démarche, pas seulement des diagrammes appris un par un. La voici, dans l'ordre que suivent les professionnels. Étape 1 : lire le besoin (cahier des charges, interview du client) et souligner les noms — ce sont des classes candidates — et les verbes d'action — des cas d'utilisation candidats. Étape 2 : fixer la frontière du système et lister les acteurs, puis construire le diagramme de cas d'utilisation et faire valider cette vue par le client : c'est le diagramme qu'il comprend le mieux. Étape 3 : construire le diagramme de classes en extrayant les concepts du domaine, leurs attributs, et en justifiant chaque multiplicité par une phrase métier. Étape 4 : choisir deux ou trois scénarios critiques et les dérouler en diagrammes de séquence. Étape 5 : vérifier la cohérence globale (voir le paragraphe suivant) avant de passer au code.

L'étape de cohérence est celle qui distingue un projet réussi d'un empilement de dessins. Trois contrôles systématiques : (a) chaque message d'un diagramme de séquence doit correspondre à une opération déclarée dans la classe receveuse du diagramme de classes — si GestionEmprunts appelle livre.estDisponible(), alors Livre doit posséder l'opération estDisponible() ; (b) chaque cas d'utilisation doit être « réalisable » : il mobilise des classes et des acteurs qui existent dans les autres diagrammes ; (c) chaque classe du diagramme de classes doit servir quelque part —

une classe qu'aucun scénario n'utilise est soit inutile, soit le signe d'un scénario oublié. Sur BiblioIAM, ce contrôle a par exemple fait apparaître l'opération clore(date) sur Emprunt, oubliée dans la première version du diagramme de classes mais indispensable au scénario de retour.

Dernière leçon, la plus importante : un modèle n'est pas un monument, c'est un outil de conversation. Il doit rester lisible par un être humain en cinq minutes, utiliser le vocabulaire du métier (les utilisateurs disent « adhérent », pas « user_entity »), et évoluer à chaque découverte. Mieux vaut un modèle imparfait mais discuté et corrigé avec le client, qu'un modèle exhaustif dessiné seul et jamais relu. Pour ton projet de fin de cours, on attendra donc : un diagramme de cas d'utilisation validé, un diagramme de classes aux multiplicités justifiées, deux diagrammes de séquence cohérents, et le squelette de code généré correspondant — exactement la chaîne que ce cours a construite, leçon après leçon, sur BiblioIAM.

L'essentiel
Démarche complète : besoin → acteurs et cas d'utilisation (validation client) → diagramme de classes (multiplicités justifiées) → scénarios en séquence → contrôles de cohérence (tout message = une opération existante ; tout cas réalisable ; toute classe utilisée) → squelettes de code. Un modèle est un outil de communication vivant, pas une œuvre définitive.

Exemple — Projet BiblioIAM : mini cahier des charges et bilan de cohérence

```
MINI CAHIER DES CHARGES (extrait)
« La bibliothèque de l'IAM prête ses livres aux
étudiants et enseignants inscrits. Un adhérent
peut avoir au plus 3 emprunts en cours, pour
14 jours. Le bibliothécaire enregistre les
emprunts et les retours au comptoir, gère les
livres et les rayons. Un livre en retard donne
lieu à une pénalité. On veut consulter le
catalogue et réserver un livre indisponible. »

LIVRABLES PRODUITS DANS CE COURS
- Cas d'utilisation : leçon 2 (acteurs, include,
extend, généralisation)
- Diagramme de classes : leçon 3 (6 classes liées,
multiplicités justifiées)
- Séquence : leçon 4 (scénario Emprunter)
- États-transitions : leçon 5 (cycle de vie du Livre)
- Squelette Java : leçon 6 (fidèle à la leçon 3)

CONTRÔLES DE COHÉRENCE PASSÉS
[OK] peutEmprunter() appelé en séquence (msg 4)
-> déclaré dans Etudiant (classes)
[OK] estDisponible() appelé (msg 5)
-> déclaré dans Livre
[OK] « create » d'Emprunt (msg 6)
-> classe Emprunt présente, liée à Personne
et à Livre avec multiplicité 1 des deux côtés
[OK] pénalité du cahier des charges
-> transition En retard / calculerPénalité
(diagramme d'états du Livre)
[OK] « au plus 3 emprunts »
-> garde de peutEmprunter() : size() < 3
```

À retenir

• Ordre de travail : cas d'utilisation pour valider le besoin, classes pour la structure, séquences pour valider les scénarios, code en dernier.

• Les trois contrôles de cohérence (messages↔opérations, cas réalisables, classes toutes utilisées) attrapent la majorité des erreurs de modélisation.

• Un bon modèle parle le langage du métier, tient sur une page lisible et change dès que le besoin change : c'est un document vivant.

Exercice

Une coopérative veut une application pour gérer les commandes de repas d'une cantine : les élèves commandent un plat la veille, la cuisine prépare selon les commandes, le caissier encaisse au moment du retrait. Sans dessiner les diagrammes en entier, produis : 1) la liste des acteurs ; 2) quatre cas d'utilisation avec leur acteur principal ; 3) trois classes candidates avec deux attributs chacune ; 4) une incohérence typique qu'un contrôle de cohérence pourrait révéler dans ce projet, et comment la corriger.

✔ Corrigé
1) Acteurs : Élève (principal), Cuisinier, Caissier ; éventuellement Responsable de la cantine pour les statistiques. 2) Cas : « Commander un plat » (Élève), « Consulter la liste des commandes du jour » (Cuisinier), « Encaisser le retrait » (Caissier), « Publier le menu de la semaine » (Responsable ou Cuisinier). « Commander un plat » inclurait logiquement « S'authentifier » en <<include>>. 3) Classes candidates : Plat (− nom : String, − prix : int) ; Commande (− dateCommande : Date, − etat : String) ; Élève (− numeroCarte : String, − classe : String). L'association naturelle : Élève 1 — 0..* Commande ; Commande * — 1 Plat (chaque commande concerne un plat dans cette version simplifiée). 4) Incohérence typique : le diagramme de séquence du scénario « Encaisser le retrait » fait appeler commande.marquerRetiree() alors que la classe Commande ne déclare que l'attribut etat et aucune opération de retrait ; ou bien le cas « Publier le menu » existe sans qu'aucune classe Menu ne figure au diagramme de classes. Correction : ajouter l'opération (ou la classe) manquante au diagramme de classes — c'est le contrôle (a) messages↔opérations qui révèle le trou, et c'est exactement pour cela qu'on modélise avant de coder.

Travaux dirigés (TD)

TD 1 — Cas d'utilisation d'un système de location de voitures

Une agence de location de voitures veut informatiser son accueil. Le client consulte les véhicules disponibles et réserve un véhicule pour une période ; pour réserver, il doit être identifié (compte vérifié par l'agence). À la prise du véhicule, l'agent effectue systématiquement un contrôle de l'état du véhicule avec le client ; le client peut, s'il le souhaite, souscrire une assurance complémentaire au moment de la réservation (ce n'est jamais obligatoire). L'agent gère aussi le parc (ajout, retrait de véhicules) et clôture les locations au retour en vérifiant l'état du véhicule. Travail demandé : 1) liste des acteurs et de leurs spécialisations éventuelles ; 2) diagramme de cas d'utilisation en texte structuré, avec include et extend correctement placés (sens des flèches et conditions) ; 3) une phrase de justification pour chaque relation choisie.

✔ Corrigé du TD 1
Acteurs : Client (principal), Agent d'agence (principal pour la gestion). Pas de généralisation nécessaire ici, sauf si l'on distingue Client et Agent comme spécialisations d'un acteur « Utilisateur » pour le cas commun « S'authentifier » — acceptable mais non exigé. Cas d'utilisation : Consulter les véhicules (Client) ; Réserver un véhicule (Client) ; Prendre le véhicule (Client, Agent) ; Clôturer la location (Agent) ; Gérer le parc de véhicules (Agent) ; S'identifier (cas inclus). Relations : Réserver --- <<include>> ---> S'identifier : l'identification est systématique, exigée par l'énoncé (« doit être identifié »), donc toujours — flèche du cas de base vers le cas inclus. Prendre le véhicule --- <<include>> ---> Contrôler l'état du véhicule : le contrôle est effectué SYSTÉMATIQUEMENT à la prise, donc include également. Souscrire une assurance complémentaire --- <<extend>> ---> Réserver un véhicule, condition [client le souhaite] : l'assurance est optionnelle et jamais obligatoire, c'est le portrait robot de l'extend, et la flèche part du cas d'extension vers le cas de base. Erreurs fréquentes sanctionnées : inverser la flèche de l'extend (la faire partir de Réserver) ; mettre l'assurance en include sous prétexte qu'elle est « proposée à chaque fois » — être proposée n'est pas être souscrite ; relier Clôturer la location au Client comme acteur principal, alors que l'énoncé la confie à l'agent.

TD 2 — Diagramme d'activité avec couloirs : le retour d'un livre à BiblioIAM

Décris en texte structuré le diagramme d'activité du processus « Retourner un livre », avec deux couloirs (Bibliothécaire / Système). Le processus : le bibliothécaire scanne le code-barres du livre ; le système retrouve l'emprunt en cours ; si l'emprunt est en retard, le système calcule la pénalité et le bibliothécaire encaisse le paiement ; dans tous les cas, le système clôture l'emprunt, PUIS, en parallèle, il remet l'exemplaire en stock ET envoie une notification de confirmation à l'adhérent ; le processus se termine quand les deux branches parallèles sont achevées. Travail demandé : la liste ordonnée des actions par couloir, le losange de décision avec ses gardes, et l'emplacement exact de la fourche et de la jonction.

✔ Corrigé du TD 2
Couloir Bibliothécaire : (début) Scanner le code-barres du livre → [plus tard, seulement si retard] Encaisser la pénalité. Couloir Système : Rechercher l'emprunt en cours → Décision [emprunt en retard ?] : branche [oui] → Calculer la pénalité → (retour au couloir Bibliothécaire : Encaisser) ; branche [non] → directement à la fusion ; Fusion → Clôturer l'emprunt (dateRetour = aujourd'hui) → FOURCHE (barre épaisse) : branche A : Remettre l'exemplaire en stock (nbExemplaires + 1) ; branche B : Envoyer la notification à l'adhérent ; les deux branches sont SIMULTANÉES, sans ordre imposé entre elles → JONCTION (barre épaisse) : on n'avance que lorsque A ET B sont terminées → (fin). Points de notation qui font la différence : la décision n'a qu'une entrée et des sorties gardées complémentaires ([en retard] / [sinon]) ; l'encaissement est une action du couloir Bibliothécaire même s'il est déclenché par le calcul du Système — le couloir dit QUI fait, pas qui décide ; la jonction attend les DEUX branches : envoyer la notification avant la remise en stock, ou l'inverse, ne termine pas le processus tant que l'autre branche n'est pas finie. Placer la fourche avant la clôture serait une erreur : on ne doit ni remettre en stock ni notifier un emprunt qui n'est pas encore clos.

Travaux pratiques (TP)

TP 1 — Modéliser la gestion d'une boutique avec PlantUML ou draw.io

Par binôme, modélisez l'application « BoutiqueIAM » : une boutique de quartier veut suivre ses produits et ses ventes. Le vendeur enregistre des ventes (plusieurs produits par vente, avec quantités), consulte le stock et reçoit une alerte quand un produit passe sous le seuil de 5 unités ; le gérant ajoute les produits, fixe les prix et consulte le chiffre d'affaires du jour. Livrables : 1) un diagramme de cas d'utilisation (acteurs, 5 cas minimum, au moins un include et un extend justifiés) ; 2) un diagramme de classes (Produit, Vente, LigneVente, Vendeur, Gérant ou une organisation équivalente justifiée) avec multiplicités argumentées ; 3) un diagramme de séquence du scénario « Enregistrer une vente » ; 4) les squelettes Java ou Python des classes du diagramme. Réalise les diagrammes dans PlantUML (texte) ou draw.io, puis exporte-les en images jointes au rapport.

✔ Attendus du TP 1
Attendus (une organisation équivalente argumentée est acceptée) : Cas d'utilisation — acteurs Vendeur et Gérant (spécialisations possibles d'un acteur Personnel) ; cas : Enregistrer une vente (Vendeur), Consulter le stock (Vendeur, Gérant), Gérer les produits (Gérant), Consulter le chiffre d'affaires (Gérant), S'authentifier ; « Enregistrer une vente » --- <<include>> ---> « Vérifier le stock disponible » (systématique) ; « Appliquer une remise fidélité » --- <<extend>> ---> « Enregistrer une vente », condition [client fidèle] (optionnel). Classes — Produit (− reference, − nom, − prix, − stock) ; Vente (− date, + total()) ; LigneVente (− quantite, − prixVente) ; relations : Vente 1 ◆— 1..* LigneVente (composition : les lignes meurent avec la vente) ; LigneVente * — 1 Produit (association simple : le produit survit à la vente) ; Vendeur 1 — 0..* Vente (qui a enregistré quoi). Séquence « Enregistrer une vente » : Vendeur → Interface : saisirVente() ; boucle [pour chaque produit] : vérifier stock, créer une LigneVente ; fin : calculer et afficher le total ; fragment alt [stock insuffisant] : message d'erreur et produit retiré. Squelettes : les champs reprennent les attributs avec leurs visibilités, Vente contient une List<LigneVente> créée dans son constructeur (composition), chaque LigneVente référence un Produit. Barème indicatif : cohérence cas/classes (1/3), multiplicités justifiées (1/3), fidélité du squelette au diagramme (1/3).

Exercices corrigés supplémentaires

Exercice 1

Le diagramme de classes suivant contient trois erreurs de modélisation. Énoncé des relations dessinées par un étudiant pour BiblioIAM : (a) « Emprunt contient ses dates, donc Emprunt est en composition vers Personne, losange plein du côté d'Emprunt » ; (b) « Personne 0..* — 0..* Emprunt, car un adhérent peut emprunter plusieurs livres » ; (c) « Étudiant hérite de la classe Emprunt, car ce sont les étudiants qui empruntent ». Trouve et corrige les trois erreurs, en justifiant chaque correction.

✔ Corrigé
Erreur (a) : le raisonnement et le sens sont faux deux fois. La composition est une relation tout→partie entre OBJETS, pas entre une classe et ses attributs : les dates sont de simples attributs d'Emprunt, elles ne forment pas une relation. Et quand composition il y a, le losange se place du côté du TOUT, jamais de la partie. Il n'y a ici aucune composition : Personne—Emprunt est une association simple. Erreur (b) : la multiplicité côté Personne, lue depuis Emprunt, doit être exactement 1 : chaque emprunt est réalisé par UN SEUL adhérent identifié (on doit toujours savoir qui a emprunté). « Un adhérent a plusieurs emprunts » se traduit par 0..* du côté Emprunt en lisant depuis Personne, ce qui donne Personne 1 — 0..* Emprunt. Écrire 0..* des deux côtés autoriserait un emprunt sans adhérent ou partagé entre plusieurs adhérents : contraire au métier. Erreur (c) : l'héritage exige un « est-un » ; un étudiant n'EST PAS un emprunt. La relation correcte entre Étudiant (via Personne) et Emprunt est l'association « réalise ». Hériter d'Emprunt donnerait à chaque étudiant des dates d'emprunt comme attributs personnels — une absurdité qui se détecte en prononçant la phrase « un Étudiant est un Emprunt » : si elle sonne faux, la généralisation est fausse.

Exercice 2

Pour chaque besoin, choisis LE diagramme le plus adapté parmi : cas d'utilisation, classes, séquence, activité, états-transitions — et justifie en une phrase. (1) « Le directeur veut voir, en une page, qui se servira du logiciel de paie et pour quelles fonctions. » (2) « On veut vérifier dans quel ordre l'interface, le contrôleur et la base échangent leurs appels pendant la connexion d'un utilisateur. » (3) « Une facture passe par les situations : émise, relancée, payée, contentieuse, soldée ; on veut connaître les passages autorisés. » (4) « Le processus de recrutement comporte des étapes, des choix, et deux branches qui avancent en même temps (vérification des diplômes et visite médicale). »

✔ Corrigé

(1) Cas d'utilisation : la demande porte sur les acteurs et les fonctionnalités offertes, vue externe en une page pour un non-informaticien. (2) Diagramme de séquence : la question est explicitement chronologique — l'ordre des appels entre objets d'un scénario, c'est sa spécialité (le diagramme de communication montrerait les mêmes échanges mais en perdant la lecture du temps de haut en bas). (3) États-transitions : on décrit le cycle de vie d'UN objet (la facture), ses situations durables et les transitions autorisées entre elles ; tout passage non dessiné sera impossible. (4) Diagramme d'activité : processus multi-étapes avec décision et surtout parallélisme — la fourche et la jonction y expriment exactement les deux branches simultanées qui doivent toutes deux finir avant la suite. Réflexe général : qui/quoi → cas d'utilisation ; avec quoi (structure) → classes ; dans quel ordre → séquence ; quelles étapes/processus → activité ; quelles situations d'un objet → états-transitions.

Exercice 3

Petit exercice de traduction modèle → code. Diagramme donné : Voiture { − marque : String ; − places : int } ; Moteur { − puissance : int } ; Voiture 1 ◆— 1 Moteur (composition) ; Conducteur { − nom : String } ; Conducteur 1 — 0..* Voiture (« possède »). Écris les squelettes Java des trois classes (champs et constructeur utile seulement), en respectant exactement le diagramme.

✔ Corrigé

class Moteur { private int puissance; } — aucune relation ne part de Moteur, c'est une coquille simple. class Voiture { private String marque; private int places; private Moteur moteur; public Voiture() { this.moteur = new Moteur(); } } — la composition 1 ◆— 1 se traduit par un attribut moteur (multiplicité 1 : pas une collection) créé dans le constructeur de Voiture, car la partie naît et meurt avec le tout et ne doit pas être partagée. class Conducteur { private String nom; private List<Voiture> voitures = new ArrayList<>(); } — l'association « possède » de multiplicité 0..* côté Voiture devient une collection dans Conducteur ; côté Voiture, on ne met un attribut Conducteur que si l'association doit être navigable dans les deux sens, ce qui n'est pas demandé ici. Erreurs classiques : List<Moteur> dans Voiture (la multiplicité est 1, pas *) ; recevoir le moteur en paramètre du constructeur (cela en ferait un objet partagé venu de l'extérieur, contraire à la composition) ; oublier la collection dans Conducteur en mettant un attribut simple Voiture (un conducteur peut en posséder plusieurs).

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Quel est le but premier d'un modèle UML ?
   A) Remplacer définitivement le code source
   B) Communiquer et concevoir le système avant de le coder
   C) Générer automatiquement tout le logiciel sans programmeur
2. Qu'est-ce qu'UML, exactement ?
   A) Un langage de programmation orienté objet
   B) Une méthode de gestion de projet
   C) Un langage de modélisation graphique standardisé par l'OMG
3. Dans un diagramme de cas d'utilisation, un acteur représente…
   A) Un rôle externe au système qui interagit avec lui
   B) Une personne précise, nommée par son nom
   C) Une classe interne du système
4. « Emprunter un livre » <<include>> « S'authentifier » signifie que…
   A) L'authentification est facultative
   B) L'authentification fait toujours partie de l'emprunt
   C) On peut emprunter sans jamais s'authentifier
5. La relation <<extend>> se reconnaît à ce que le comportement ajouté…
   A) Se produit à chaque exécution du cas de base
   B) Est obligatoire pour tous les acteurs
   C) Est optionnel et ne s'insère que si une condition est remplie
6. Dans « Personne 1 — 0..* Emprunt », la multiplicité 0..* signifie que…
   A) Chaque emprunt appartient à plusieurs personnes
   B) Une personne peut n'avoir aucun emprunt, ou en avoir plusieurs
   C) L'emprunt est interdit aux personnes
7. Un losange plein sur une association (composition) implique que…
   A) La partie survit toujours à la disparition du tout
   B) La partie n'appartient qu'à un seul tout et est détruite avec lui
   C) La relation est temporaire, le temps d'un appel
8. Une flèche à triangle vide reliant Étudiant à Personne représente…
   A) Une agrégation : l'étudiant contient la personne
   B) Une dépendance : l'étudiant utilise la personne
   C) Une généralisation : un étudiant EST une personne et en hérite

9. Dans un diagramme de séquence, un message dessiné avec une flèche pleine est…

A) Un message de retour

B) Synchrone : l'émetteur attend la réponse avant de continuer

C) Asynchrone : l'émetteur continue sans attendre

10. Une association de multiplicité * devient, dans un squelette Java…

A) Un attribut simple du type de la classe cible

B) Une énumération de constantes

C) Une collection, par exemple une List de la classe cible

Corrigés du QCM

1. Réponse B — Le modèle est un outil de réflexion et de communication : on y valide le besoin et la structure à moindre coût, avant d'investir dans le code.

2. Réponse C — UML (Unified Modeling Language) est une notation standard de modélisation ; il ne s'exécute pas comme un langage et n'impose aucune méthode de travail.

3. Réponse A — Un acteur est un rôle (Bibliothécaire, Étudiant, mais aussi un système externe), jamais un individu nommé ni un élément interne du logiciel.

4. Réponse B — L'inclusion est systématique : à chaque exécution du cas de base, le cas inclus se produit. La flèche part du cas de base vers le cas inclus.

5. Réponse C — C'est le critère décisif : parfois et sous condition = extend ; toujours = include. Et la flèche d'extension part du cas d'extension vers le cas de base.

6. Réponse B — 0..* se lit depuis l'objet d'en face : pour UNE personne, le nombre d'emprunts liés va de zéro à plusieurs. L'autre extrémité (1) garantit qu'un emprunt a exactement un adhérent.

7. Réponse B — Composition = lien fort : partie exclusive, créée et détruite avec le tout. Le losange vide (agrégation) est le lien faible ; la relation temporaire relève de la dépendance.

8. Réponse C — Le grand triangle vide est la signature de la généralisation (héritage) : la sous-classe est une sorte de la classe mère et hérite de ses attributs et opérations.

9. Réponse B — La pointe de flèche pleine marque un appel synchrone : l'émetteur se bloque jusqu'à la réponse (flèche pointillée). L'asynchrone se dessine avec une pointe ouverte et n'attend rien — par exemple l'envoi d'un e-mail de confirmation.

10. Réponse C — Plusieurs objets liés = il faut un conteneur : une List (ou un Set selon le cas). Multiplicité 1, en revanche, donne un attribut simple.

Résumé — les notions importantes

• Modéliser avant de coder : UML est la notation standard (OMG) pour décrire un système en 14 diagrammes, structurels et comportementaux ; ce cours en pratique six.

• Cas d'utilisation : des acteurs-rôles, des fonctionnalités en « verbe + complément », une frontière de système ; include = toujours (flèche base → inclus), extend = parfois sous condition (flèche extension → base).

• Diagramme de classes : attributs typés et opérations avec visibilités (+, −, #), classes abstraites ; chaque multiplicité se justifie par une phrase du domaine, lue dans les deux sens.

• Généralisation = « est-un » hérité ; agrégation = « a-un » faible (losange vide, parties indépendantes) ; composition = « a-un » fort (losange plein, partie exclusive détruite avec le tout) ; dépendance = usage ponctuel.

• Séquence : un scénario, le temps de haut en bas ; synchrone = flèche pleine et attente, asynchrone = flèche ouverte, retour en pointillé ; fragments alt/opt/loop/ref. Communication : mêmes échanges vus par les liens, messages numérotés et imbriqués.

• Activité : un processus (actions, décisions gardées, fourche/jonction, couloirs). États-transitions : la vie d'un objet (états, transitions événement [garde] / action), qui interdit tout chemin non dessiné.

• Modèle → code : classe → classe, attribut → champ, multiplicité 1 → attribut simple, multiplicité * → collection, composition → création dans le tout, dépendance → paramètre ; puis on vérifie la cohérence entre tous les diagrammes.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — Modéliser avec UML

Chaîne : Comprendre le Code · Langue : Français · Niveau : Intermédiaire

Une présentation claire de l'intérêt d'UML : pourquoi modéliser, en quoi le langage est un standard, et un tour d'horizon des diagrammes structurels et comportementaux les plus utilisés. Parfaite pour replacer ce cours dans une démarche de conception complète avant d'attaquer les diagrammes un par un.

https://www.youtube.com/watch?v=R5qq-sHaiz8

Leçon 2 — Diagramme des Cas d'Utilisation : Cours Complet et Exercice Corrigé sur Astah UML

Chaîne : Prof. Fatimazahra BARRAMOU · Langue : Français · Niveau : Débutant à intermédiaire

Le cours complet sur les cas d'utilisation : définition des acteurs, relations d'inclusion, d'extension et de généralisation, puis un exercice corrigé construit pas à pas dans l'outil Astah. À regarder en parallèle de la leçon 2 pour vérifier le sens de chaque flèche.

https://www.youtube.com/watch?v=NeYSBDMlVvY

Leçon 2 — Cours UML #1 | Diagramme de Cas D'utilisation | Avec Des Exercices

Chaîne : Yassine ENNAJEM · Langue : Français · Niveau : Débutant

Un premier cours UML entièrement consacré au diagramme de cas d'utilisation, avec des exercices d'application. Utile en révision rapide avant le TD de ce cours, notamment pour distinguer les acteurs des éléments internes du système.

https://www.youtube.com/watch?v=aIbzcH_ItQo

Leçon 3 — Diagramme de Classe UML - Exercice Corrigé sur Astah UML

Chaîne : Prof. Fatimazahra BARRAMOU · Langue : Français · Niveau : Débutant à intermédiaire

Un exercice complet de diagramme de classes corrigé en vidéo, construit dans Astah UML à partir d'un cahier des charges. Observe surtout comment les multiplicités sont choisies et justifiées, puis compare avec le diagramme de classes de BiblioIAM de la leçon 3.

https://www.youtube.com/watch?v=y1oThKZb7vc

Leçon 4 — Diagramme de Séquence UML : Cours complet et Exercice sur Astah UML

Chaîne : Prof. Fatimazahra BARRAMOU · Langue : Français · Niveau : Débutant à intermédiaire

Le diagramme de séquence de A à Z : lignes de vie, messages synchrones et asynchrones, messages de création et de destruction, fragments alt, loop, opt et par. Le complément idéal de la leçon 4, avec une démonstration complète dans l'outil.

https://www.youtube.com/watch?v=fKUn5f-kSAg

Sources et références

- Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.
- OMG (Object Management Group) — Spécification officielle UML (omg.org/spec/UML) : le document normatif qui définit la notation, pour trancher tout doute sur un symbole.
- Pascal Roques — « UML 2 par la pratique » (Eyrolles) : l'ouvrage francophone de référence pour apprendre UML par des études de cas, du cas d'utilisation au diagramme de classes.
- Documentation PlantUML (plantuml.com/fr) — pour dessiner tes diagrammes en texte, comme dans le TP de ce cours, et les exporter en images.
- Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !